

LAB 08

Consolidated Semantic Model & Transactional Write-Backs

Direct Lake · XMLA / Tabular Editor Governance · Fabric Apps · GraphQL · ACID · Eventstream

MODULE

Semantic Modelling & Analytics
Engineering

DURATION

3 - 4 hours

DIFFICULTY

Advanced

Lab Goal and Problem Statement

LAB GOAL

Build and govern a small consolidated enterprise semantic model in Direct Lake mode, then implement and validate transactional write-backs through a Fabric App, Fabric API for GraphQL, SQL / Warehouse ACID transactions, and Eventstream.

Database Mirroring is intentionally excluded: every mirroring setup, synchronisation, failover, replica and mirrored-OneLake activity has been removed. All other module coverage is retained.

PROBLEM STATEMENT

The enterprise sales team needs a governed Fabric analytics and transactional workflow: a consolidated Direct Lake semantic model for sales insights plus controlled order-approval write-backs.

Using the synthetic dataset LAB_08_Dataset_Package.zip (Sales.csv, SalesOrderApproval.csv) you will build and validate the semantic model, create business measures, identify the top region and product category, calculate total revenue and average order value, and validate controlled write-backs through the Fabric App, Fabric API for GraphQL and Warehouse SQL transactions.

The workflow must demonstrate COMMIT, ROLLBACK / ACID behaviour, concurrent-write handling and Eventstream change capture.



STEP 00 - NOT COUNTED

Prerequisites

- Microsoft Fabric workspace and capacity
- A Fabric Lakehouse with permission to load tables
- A Fabric Warehouse or Azure SQL / Fabric SQL backend with DDL / DML permissions
- Power BI / Fabric semantic-model capability with Direct Lake support
- XMLA read / write endpoint access or Tabular Editor access, if enabled by the instructor
- Fabric App capability and a Fabric API for GraphQL item, or instructor-provided equivalents
- Fabric Eventstream capability
- Download the separate LAB_08_Dataset_Package.zip supplied with this lab

CONTROLLED TEST RECORD

1001

Use OrderID / SaleID 1001 as the only write-back test record. Do not use production identifiers.



STEP 01

Load the Lab Dataset

WHERE

Fabric Workspace → Lakehouse / Warehouse

WHAT TO CLICK / OPEN

Open the assigned Lakehouse and Warehouse. Use the supplied dataset package (LAB_08_Dataset_Package.zip).

WHAT TO CREATE / SELECT / CONFIGURE

Load Sales.csv into the Lakehouse as a table named Sales. Run Warehouse_Setup.sql in the Warehouse to create Sales and SalesOrderApproval.

WHAT TO VALIDATE

Both tables exist and contain the supplied synthetic data. No production data is modified.

The screenshot displays the Microsoft Fabric Lakehouse interface. The top navigation bar includes the 'Fabric' logo, a search bar, and a notification about the free trial ending in 0 days. The left sidebar shows the 'Explorer' pane with a tree view of the workspace, including 'sample_data', 'Tables', 'dbo', and 'Files'. The main area shows a preview of the 'Sales.csv' file, which contains 13 rows of synthetic data. The data is presented in a table view with columns for SaleID, CustomerID, CustomerName, Region, Product, ProductCategory, OrderDate, Quantity, and Revenue.

	() _c0	ABC _c1	ABC _c2	ABC _c3	ABC _c4	ABC _c5	() _c6	() _c7	() _c8
1	SaleID	CustomerID	CustomerName	Region	Product	ProductCategory	OrderDate	Quantity	Revenue
2	1001	C001	Alice	Pune	Laptop	Electronics	2026-08-01	2	2400.0
3	1002	C002	Bob	Mumbai	Monitor	Electronics	2026-08-02	3	900.0
4	1003	C003	Charlie	Delhi	Office Chair	Furniture	2026-08-03	5	1250.0
5	1004	C001	Alice	Pune	Laptop	Electronics	2026-08-05	1	1200.0
6	1005	C002	Bob	Mumbai	Desk	Furniture	2026-08-07	2	700.0
7	1006	C003	Charlie	Delhi	Keyboard	Accessories	2026-08-09	4	320.0
8	1007	C001	Alice	Pune	Laptop	Electronics	2026-08-12	1	1200.0
9	1008	C002	Bob	Mumbai	Monitor	Electronics	2026-08-15	2	600.0
10	1009	C003	Charlie	Delhi	Desk	Furniture	2026-08-18	3	1050.0
11	1010	C001	Alice	Pune	Mouse	Accessories	2026-08-20	5	250.0
12	1011	C002	Bob	Mumbai	Laptop	Electronics	2026-08-22	1	1200.0
13	1012	C003	Charlie	Delhi	Monitor	Electronics	2026-08-25	1	300.0

02

STEP 02

Validate Source Schema and Baseline

WHERE

Fabric Warehouse → SQL Query Editor

WHAT TO CLICK / OPEN

Open New SQL query.

WHAT TO CREATE / SELECT / CONFIGURE

Inspect the columns, types, row counts, and the safe write-back row 1001.

WHAT TO VALIDATE

Sales contains SaleID, CustomerID, CustomerName, Region, Product, ProductCategory, OrderDate, Quantity, Revenue. SalesOrderApproval contains OrderID, OrderStatus, ApprovedBy, ApprovalTimestamp, Version.

RUN

```
SELECT
    TABLE_SCHEMA,
    TABLE_NAME,
    COLUMN_NAME,
    DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'sales'
ORDER BY ORDINAL_POSITION;
```

INFORMATION_SCHEMA.COLUMNS query returning the Sales column list and data types

The screenshot displays the Microsoft Fabric web interface. The top navigation bar includes the 'Fabric' logo, a search bar, and a notification about the free trial ending in 58 days. The left sidebar shows the 'Explorer' pane with a tree view of the workspace, including 'Warehouses', 'sample_data', 'Schemas', 'Security', 'Queries', 'My queries', and 'Shared queries'. The 'SQL query 1' is selected under 'My queries'.

The main area shows the SQL query editor with the following query:

```
1 SELECT
2     TABLE_SCHEMA,
3     TABLE_NAME,
4     COLUMN_NAME,
5     DATA_TYPE
6 FROM INFORMATION_SCHEMA.COLUMNS
7 WHERE TABLE_NAME IN ('sales', 'SalesOrderApproval')
8 ORDER BY TABLE_NAME, ORDINAL_POSITION;
```

Below the query editor, the 'Results' tab is active, displaying a table with 9 rows and 4 columns: TABLE_SCHEMA, TABLE_NAME, COLUMN_NAME, and DATA_TYPE. The data is as follows:

TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	DATA_TYPE
dbo	sales	SaleID	int
dbo	sales	CustomerID	varchar
dbo	sales	CustomerName	varchar
dbo	sales	Region	varchar
dbo	sales	Product	varchar
dbo	sales	ProductCategory	varchar
dbo	sales	OrderDate	date
dbo	sales	Quantity	int
dbo	sales	Revenue	float

Explorer view of the sales table columns alongside the baseline query results

The screenshot displays the Microsoft Fabric Explorer interface. On the left, the 'Explorer' pane shows the 'sample_data' warehouse structure, including schemas, tables, and columns. The 'sales' table is expanded, showing columns: SaleID (int, null), CustomerID (varchar(8000), null), CustomerName (varchar(8000), null), Region (varchar(8000), null), Product (varchar(8000), null), ProductCategory (varchar(8000), null), OrderDate (date, null), Quantity (int, null), and Revenue (float, null). The main pane shows a SQL query named 'SQL query 1' with the following code:

```
1 SELECT COUNT(*) AS SalesRowCount
2 FROM dbo.sales;
```

Below the query editor, the 'Results' tab is active, displaying a single row of data:

	123 SalesRowCount
1	12



STEP 03

Create the Direct Lake Semantic Model

WHERE

Fabric Workspace → Lakehouse → New semantic model

WHAT TO CLICK / OPEN

Select the Sales table and create / open the semantic model.

WHAT TO CREATE / SELECT / CONFIGURE

Use Direct Lake mode. Keep the model focused on Sales analytics; expose Region, ProductCategory, Product, CustomerName, OrderDate, Quantity and Revenue.

WHAT TO VALIDATE

The model opens successfully in model view and uses the Lakehouse Sales table as its Direct Lake source.

03

New semantic model dialog - name the model and tick the Sales table

New semantic model

Direct Lake semantic model name

Sales_DirectLake_Model

Workspace

labskraft-data2

Select or deselect tables for the semantic model.

Search

SQL views cannot be selected for Direct Lake on OneLake, but tables based on SQL views can be added to the semantic model later using other storage modes.

☒ Select all

☒ dbo

☒ sales

[Learn more about Direct Lake](#)

Confirm Cancel

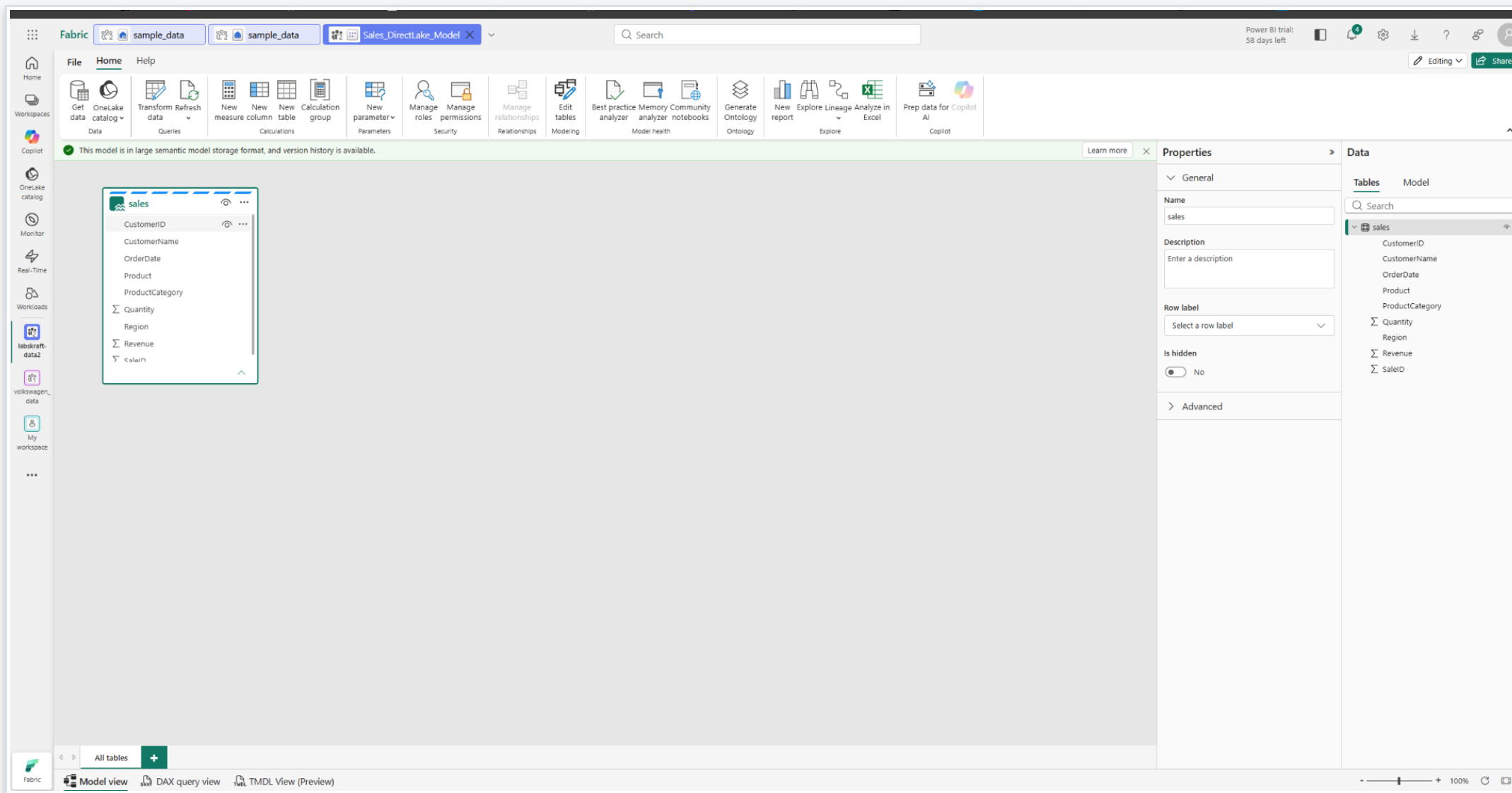
Showing 12 rows

ProductCategory	OrderDate	Quantity	Revenue
ronics	2026-08-01	2	2400
ronics	2026-08-02	3	900
ture	2026-08-03	5	1250
ronics	2026-08-05	1	1200
ture	2026-08-07	2	700
ssories	2026-08-09	4	320
ronics	2026-08-12	1	1200
ronics	2026-08-15	2	600
ture	2026-08-18	3	1050
ssories	2026-08-20	5	250
ronics	2026-08-22	1	1200
ronics	2026-08-25	1	300

Succeeded (0 sec 765 ms)

Columns 9 Rows 12

Model view open with the Sales table as the Direct Lake source





STEP 04

04

Apply Semantic Model Governance

WHERE

Semantic Model → Model view / XMLA endpoint / Tabular Editor

WHAT TO CLICK / OPEN

Open the model metadata using the XMLA read/write endpoint or Tabular Editor.

WHAT TO CREATE / SELECT / CONFIGURE

Set clear descriptions for Revenue, Region and ProductCategory; hide unnecessary technical fields; add a model version annotation such as Lab08-v1.0. Save / commit the metadata change.

WHAT TO VALIDATE

The model has readable business metadata and a traceable version marker. Do not alter source data.

[Learn more](#) X

Properties

General

Name
ProductCategory

Description
Business category assigned to the product.

Display folder
Enter the display folder

Is hidden
☒ No

Formatting

Data type
Text

Format
Text

Advanced

Data

Tables Model

Search

sales

- CustomerID
- CustomerName
- OrderDate
- Product
- ProductCategory**
- Σ Quantity
- Region
- Σ Revenue
- Σ SaleID

The screenshot shows two side-by-side panes. The left pane is titled 'Properties' and has a close button (X). It contains two sections: 'General' and 'Formatting'. Under 'General', there are fields for 'Name' (containing 'Revenue'), 'Description' (containing 'Total revenue generated from the sale.'), 'Display folder' (containing 'Enter the display folder'), and 'Is hidden' (a toggle switch set to 'No'). Under 'Formatting', there is a 'Data type' dropdown menu set to 'Decimal number' and a 'Format' field. The right pane is titled 'Data' and has a close button (»). It contains two tabs: 'Tables' (selected) and 'Model'. Below the tabs is a search bar with a magnifying glass icon and the text 'Search'. Below the search bar is a list of items under a 'sales' folder icon. The items are: 'CustomerID', 'CustomerName', 'OrderDate', 'Product', 'ProductCategory', 'Σ Quantity', 'Region', 'Σ Revenue' (highlighted with a green bar and a visibility icon), and 'Σ SaleID'.

Properties

General

Name
Revenue

Description
Total revenue generated from the sale.

Display folder
Enter the display folder

Is hidden
☒ No

Formatting

Data type
Decimal number

Format

Data

Tables Model

Search

sales

- CustomerID
- CustomerName
- OrderDate
- Product
- ProductCategory
- Σ Quantity
- Region
- Σ Revenue
- Σ SaleID

05

STEP 05

Create Core Analytics Measures

WHERE

Semantic Model → New measure

WHAT TO CLICK / OPEN

Create measures in the Sales table.

WHAT TO CREATE / SELECT / CONFIGURE

Create Total Revenue, Distinct Orders and Average Order Value. Use DISTINCTCOUNT(SaleID) for orders and DIVIDE for safe calculation.

WHAT TO VALIDATE

Measures calculate successfully against the Direct Lake data.

RUN

```
Total Revenue = SUM(Sales[Revenue])
```

```
Distinct Orders = DISTINCTCOUNT(Sales[SaleID])
```

```
Average Order Value =  
    DIVIDE([Total Revenue], [Distinct Orders])
```

New measure - Total Revenue entered in the DAX formula bar

The screenshot displays the Microsoft Fabric interface. At the top, the 'Home' tab is active, showing various toolbars for data management, modeling, and analysis. The central workspace shows a DAX formula bar with the text: `1 Total Revenue = SUM(Sales[Revenue])`. A dropdown menu is open for the 'Sales' table, listing fields: CustomerID, CustomerName, OrderDate, Product, ProductCategory, Quantity, Region, Revenue, and SaleID. On the right, the 'Properties' pane is visible, showing the 'General' tab with fields for Name (Total Revenue), Home table (Sales), and Description. Below this, there are sections for 'Formatting' (Format: General, Percentage format: No, Thousands separator: No, Decimal places: Auto) and 'Advanced'. The 'Data' pane on the far right shows the 'Tables' tab with a search bar and a list of tables: sales, CustomerID, CustomerName, OrderDate, Product, ProductCategory, Quantity, Region, Revenue, SaleID, and Total Revenue.

New measure - Distinct Orders using DISTINCTCOUNT(Sales[SalesID])

The screenshot displays the Microsoft Fabric interface. At the top, the navigation bar shows the current workspace as 'Sales_DirectLake_Model'. Below this, a ribbon menu provides options for File, Home, and Help. The central workspace contains a formula bar with the measure definition: `1 Distinct Orders = DISTINCTCOUNT(Sales[SalesID])`. A dropdown menu is open, showing the 'sales' table and its columns: CustomerID, CustomerName, OrderDate, Product, ProductCategory, Quantity, Region, Revenue, and SalesID. On the right, the 'Properties' panel is visible, showing the measure's name as 'Distinct Orders', its home table as 'sales', and a description field. Below this, the 'Formatting' section includes options for Format (Whole number), Percentage format (No), Thousands separator (No), and Decimal places (0). The 'Data' panel on the far right shows a list of tables and measures, with 'Distinct Orders' highlighted.

The screenshot displays the Microsoft Fabric interface. At the top, the 'Sales_DirectLake_Model' is selected. The 'Home' tab is active, showing a ribbon with various tools. A formula bar at the top of the workspace contains the measure definition: `1 Average Order Value = DIVIDE([Total Revenue], [Distinct Orders])`. A context menu is open over the 'sales' table, listing fields such as CustomerID, CustomerName, OrderDate, Product, ProductCategory, Quantity, Region, Revenue, and Sales. On the right, the 'Properties' pane shows the 'General' tab for the 'Average Order Value' measure, with fields for Name, Home table (set to 'sales'), Description, Display folder, and Is hidden. The 'Data' pane on the far right lists the tables and measures in the model, including 'Average Order Value'.



STEP 06

Validate Semantic Model Results

WHERE

Power BI report / semantic model → temporary validation page

WHAT TO CLICK / OPEN

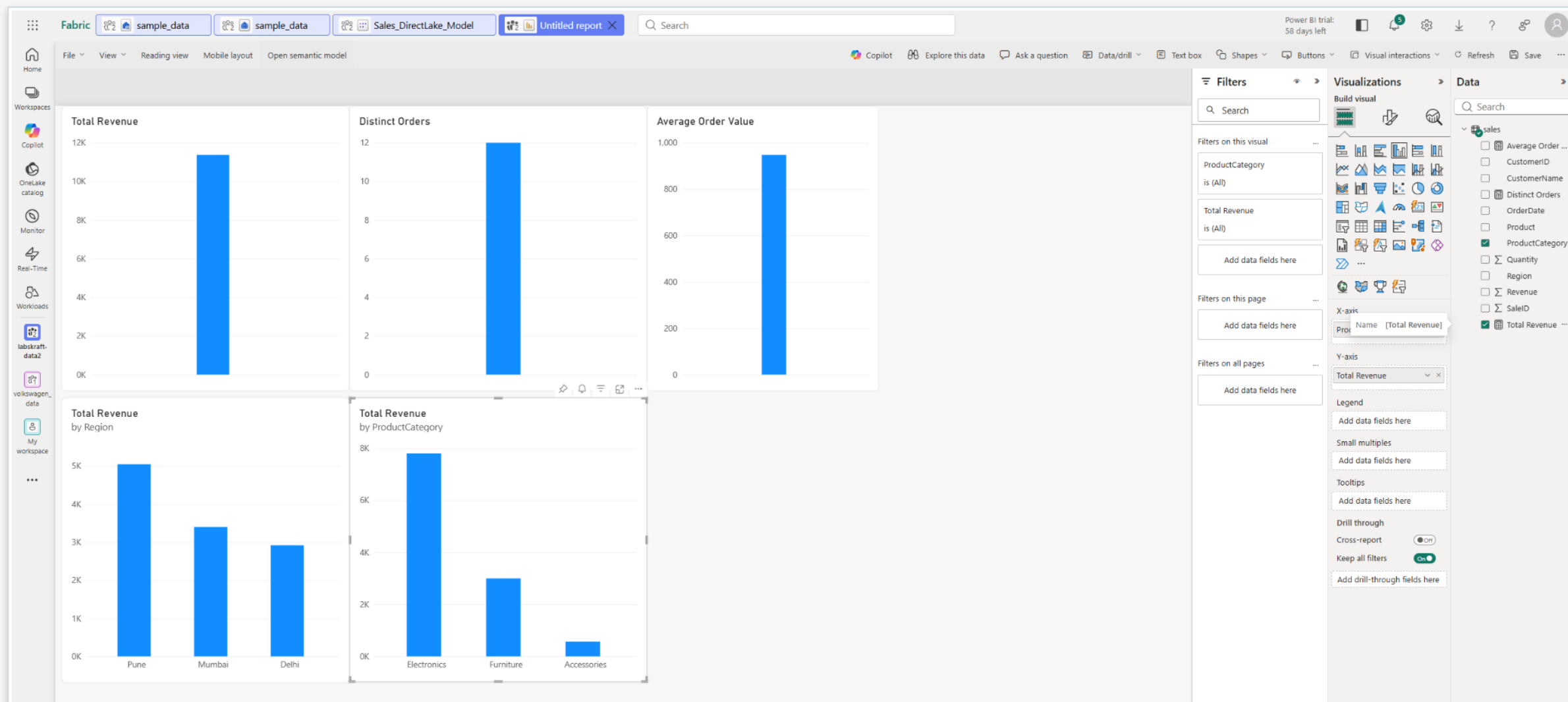
Add card / table visuals for Total Revenue, Distinct Orders, Average Order Value, Region and ProductCategory.

WHAT TO CREATE / SELECT / CONFIGURE

Rank Region and ProductCategory by Total Revenue (Sales_Business_Terminology_Report).

WHAT TO VALIDATE

Expected: Total Revenue 11,370 · Distinct Orders 12 · Average Order Value 947.50 · top Region Pune · top Category Electronics.





STEP 07

Configure Business Terminology and Model Usability

WHERE

Semantic Model → field properties / synonyms

WHAT TO CLICK / OPEN

Review field names and business-facing descriptions.

WHAT TO CREATE / SELECT / CONFIGURE

Use business terminology: CustomerID → Customer · Revenue → Sales Revenue · OrderDate → Sales Date · ProductCategory → Product Category · Region → Sales Region.

WHAT TO VALIDATE

Natural-language users can map common business terms to the intended model fields without changing source column names.

Properties

General

Name

CustomerID

Description

Customer

Display folder

Data

Tables Model

Search

sales

Average Order Value

CustomerID

CustomerName

Distinct Orders

OrderDate

Product

Properties

General

Name

OrderDate

Description

Sales Date

Display folder

Enter the display folder

Data

Tables Model

Search

sales

- Average Order Value
- CustomerID
- CustomerName
- Distinct Orders
- OrderDate
- Product
- ProductCategory

The screenshot displays the configuration interface for a data model, specifically for the 'Region' entity. The interface is divided into two main sections: 'Properties' and 'Data'.

Properties Section:

- General:**
 - Name:** Region
 - Description:** Sales Region (highlighted with a black border)
 - Display folder:** Enter the display folder
 - Is hidden:** No (toggle switch is off)
- Formatting:** (collapsed)

Data Section:

- Tables:** (selected tab)
 - Search:** Search
 - sales:** (expanded table)
 - Average Order Value
 - CustomerID
 - CustomerName
 - Distinct Orders
 - OrderDate
 - Product
 - ProductCategory
 - Quantity
 - Region** (highlighted with a grey bar)
 - Revenue
 - SaleID
 - Total Revenue
- Model:** (collapsed)



STEP 08

Validate Fabric App Write-Back Front End

WHERE

Fabric Workspace → assigned Sales Operations Fabric App

WHAT TO CLICK / OPEN

Open the app and navigate to Order Approvals. Filter to OrderID 1001.

WHAT TO CREATE / SELECT / CONFIGURE

Verify the form exposes OrderStatus and ApprovedBy and that its action maps to the approved backend / API endpoint.

WHAT TO VALIDATE

The app displays the baseline Pending record and is ready to submit a controlled write.

Detailed sub-steps with screenshots follow →

Create the Sales Operations app in the Fabric workspace

Fabric Apps sample_data sample_data Sales_DirectLake_Model Sales_Business_Terminology_Report Search

Power BI trial: 58 days left

Home Workspaces Copilot OneLake catalog Monitor Real-Time Workloads labskraft-data2 Volkswagen_data My workspace

1 Setup 2 Content* 3 Audience*

Build your app

App name *

Sales Operations

Description *

Sales Operations and Order Approvals

164 characters left

App logo

Upload Delete

App theme color

▼

Contact information

☒ Show app publisher

☐ Show items contacts from the workspace

☐ Show specific individuals or groups

Enter a name or email address

Advanced settings ▼

Next: Add content Cancel

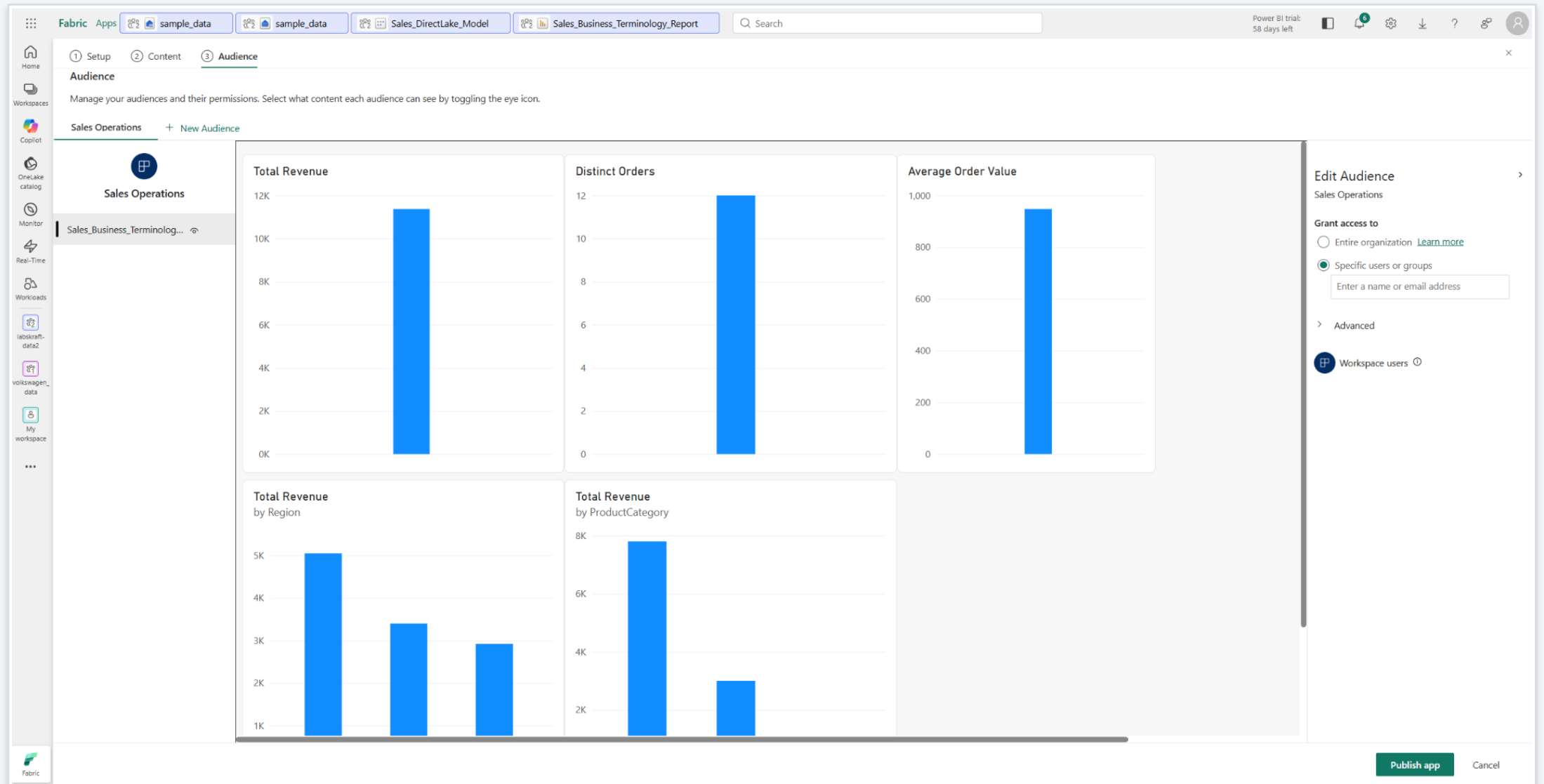
Add content - select the Sales_Business_Terminology_Report for the app

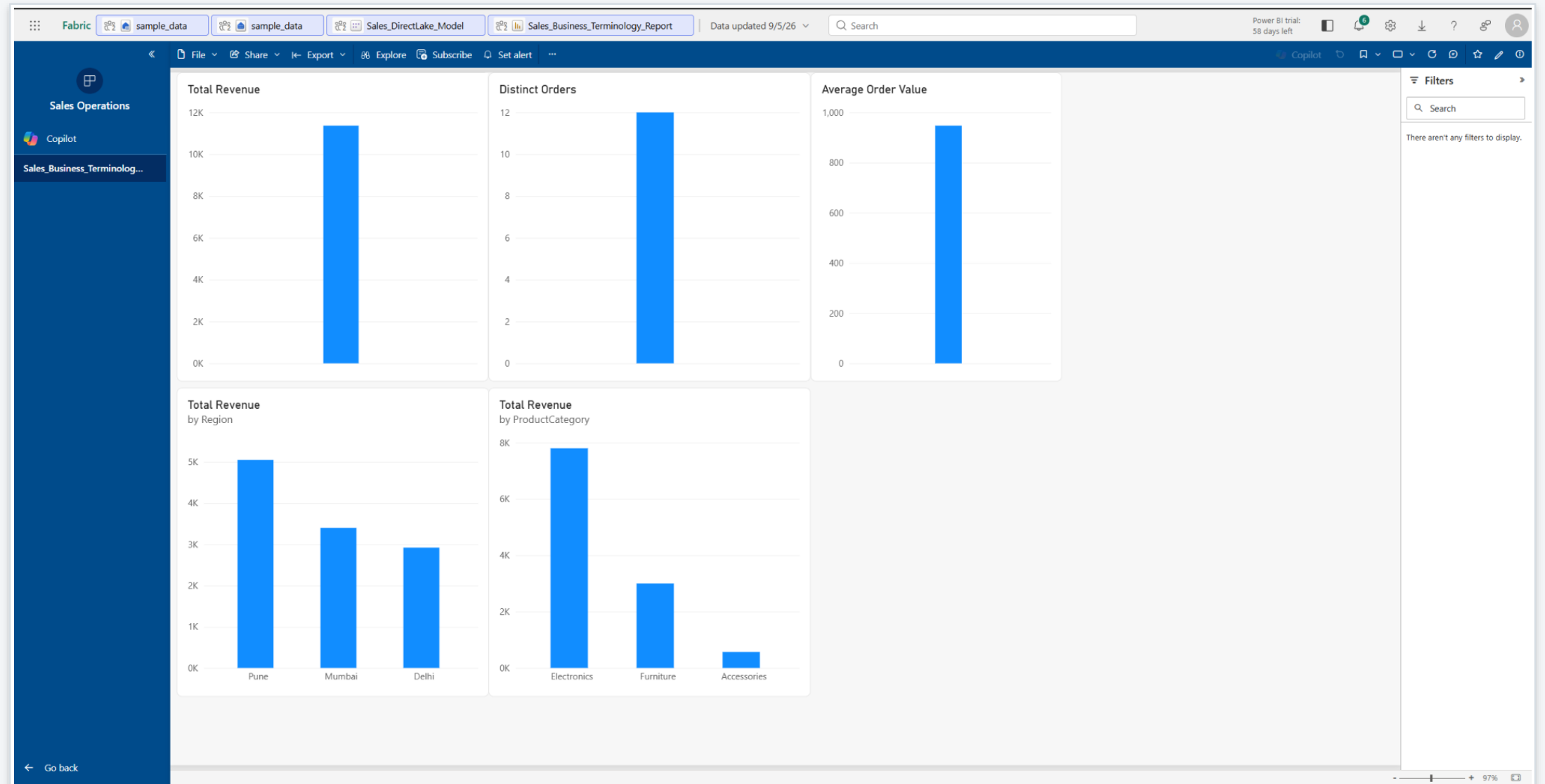
The screenshot shows a dashboard interface with a sidebar on the left and a main content area. The sidebar has a 'Sales Operations' section with a '+ Add content' button. The main content area displays a modal dialog titled 'Add content' with a close button (X) in the top right corner. The dialog contains instructions: 'Insert reports, dashboards, and workbooks directly from your workspace. You can also add website links.' Below this, there are two tabs: 'Add from workspace' (selected) and 'Add a link'. Under the 'Add from workspace' tab, there is a table with the following data:

☒		Name	Type	Owner	Refreshed
☒		Sales_Business_Terminolog	Report	labskraft-data2	9/5/26, 12:04:58 PM

At the bottom right of the dialog, there are two buttons: 'Add' (green) and 'Cancel' (white).

App content view showing the validation report inside Sales Operations





8.1 Open the Power App

- Go to make.powerapps.com
- Select the environment: KloudKraft
- Left menu → Apps
- Open Sales Order Approvals
- Click Edit

Screenshot on the next slide →

The screenshot shows the Power Apps interface. The top navigation bar is purple with the 'Power Apps' logo, a search bar, and the environment 'KloudKraft'. The left sidebar contains navigation options: Home, Create, Learn, Plans, Apps (selected), AI hub, Tables, Flows, Solutions, More, and Power Platform. The main area displays the 'Apps' list. At the top, there are buttons for 'New app', 'Import app', 'Monitor', and 'All apps'. Below these are two cards: 'Start with design' and 'Start with data'. Further down are filters for 'My apps', 'Shared with me', and 'All'. The app list table shows one app: 'Sales Order Approvals'.

Power Apps Search Environment KloudKraft

+ New app ▾ → Import app ▾ Monitor All apps Search

Apps



Start with design
Pick a layout or start from blank to shape your app's UI.



Start with data
Pick a source or create tables to power your app.

 My apps  Shared with me  All  

 Name ▾	Modified ▾	Owner ▾	Type ▾
 Sales Order Approvals	⋮ 6 hours ago	Kunal Narkhede	Canvas

8.2 Create the Text Input - TextInput3

- In the left Tree view, select Order Approvals
- Click Insert → Input → Text input
- Select the new text box
- Properties: set Name to TextInput3, set Default to "1001"
- Resize and place near the upper-left of the screen

EXPECTED RESULT ON THE CANVAS

1001

8.3 Create the Order Status label - Label4

- Insert → Text label, then select it
- Rename it to Label4
- Select the Text property in the formula bar
- Enter "OrderStatus: Pending"
- Position it underneath TextInput3

EXPECTED RESULT ON THE CANVAS

```
1001  
OrderStatus: Pending
```

8.4 Create the Approved By label - Label5

- Insert → Text label, then select it
- Rename it to Label5
- Set its Text property to "ApprovedBy:"
- Place it below Label4

EXPECTED RESULT ON THE CANVAS

```
1001  
OrderStatus: Pending  
ApprovedBy:
```

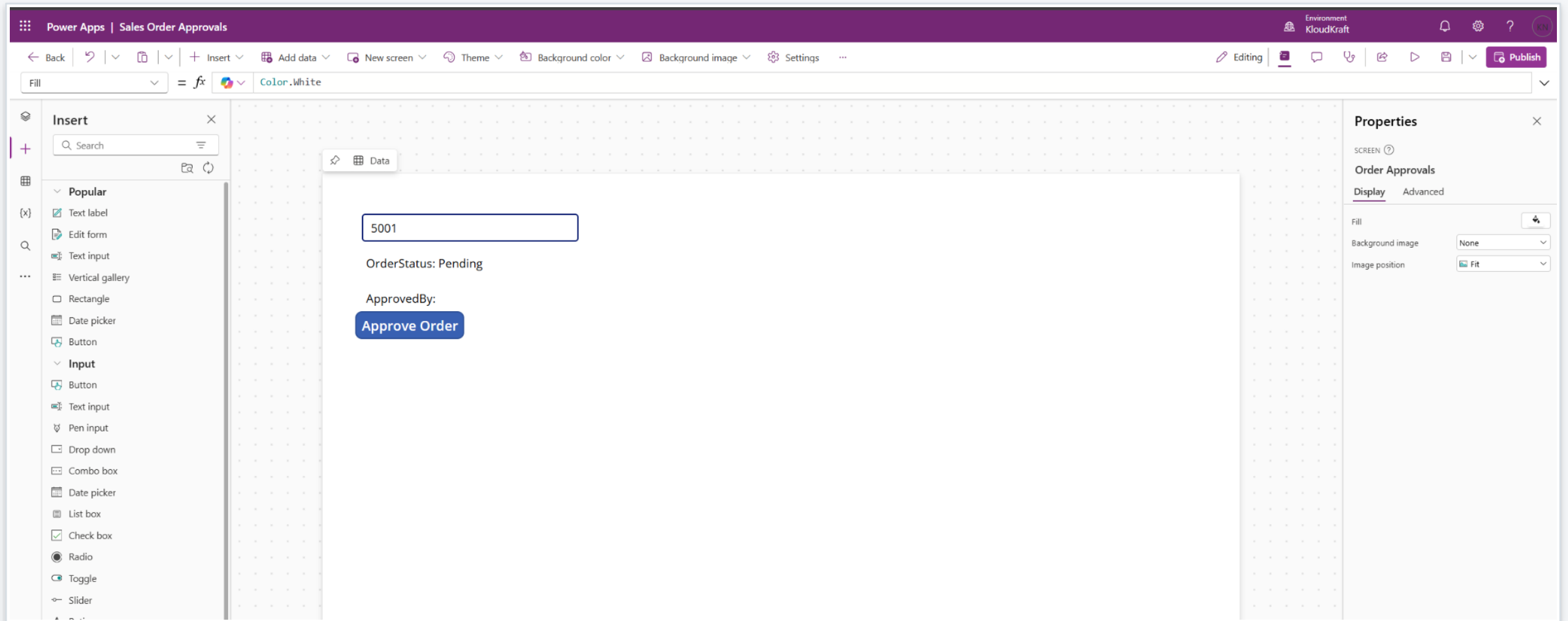
8.5 Create the Approve Order button - Button2

- Insert → Input → Button, then select it
- Rename it to Button2
- Properties: set Text to Approve Order
- Place it below Label5 and resize to match

EXPECTED RESULT ON THE CANVAS

```
1001  
OrderStatus: Pending  
ApprovedBy:  
[ Approve Order ]
```

Screenshot on the next slide →



09

STEP 09

Validate SQL Backend and Baseline Record

WHERE

Fabric Warehouse → sample_data → SQL Query Editor

WHAT TO CLICK / OPEN

Open a new SQL query.

WHAT TO CREATE / SELECT / CONFIGURE

Inspect dbo.sales and confirm the controlled test record is SaleID = 1001.

WHAT TO VALIDATE

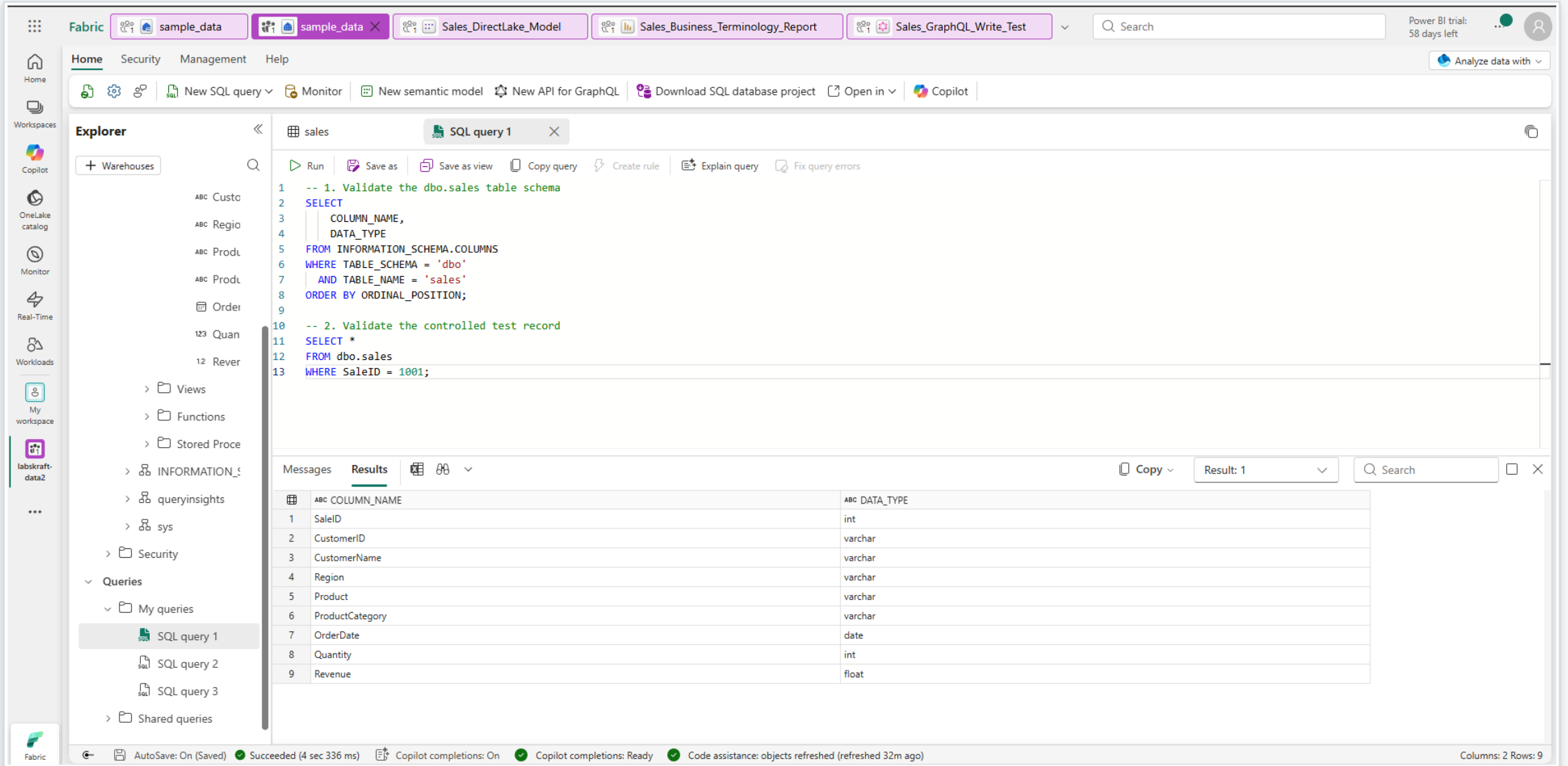
Confirm dbo.sales exists, SaleID is the record identifier, and SaleID 1001 can be queried.

RUN

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
      AND TABLE_NAME = 'sales'
ORDER BY ORDINAL_POSITION;

SELECT * FROM dbo.sales WHERE SaleID = 1001;
```

dbo.sales column list returned from INFORMATION_SCHEMA.COLUMNS



The screenshot shows the Microsoft Fabric SQL query editor interface. The top navigation bar includes tabs for 'sample_data', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', and 'Sales_GraphQL_Write_Test'. The left sidebar shows the 'Explorer' pane with a tree view of the workspace, including 'Warehouses', 'Views', 'Functions', 'Stored Proce...', 'INFORMATION_...', 'queryinsights', 'sys', 'Security', 'Queries', and 'My queries'. The main editor area displays a SQL query titled 'SQL query 1' with the following code:

```
1 -- 1. Validate the dbo.sales table schema
2 SELECT
3     COLUMN_NAME,
4     DATA_TYPE
5 FROM INFORMATION_SCHEMA.COLUMNS
6 WHERE TABLE_SCHEMA = 'dbo'
7       AND TABLE_NAME = 'sales'
8 ORDER BY ORDINAL_POSITION;
9
10 -- 2. Validate the controlled test record
11 SELECT *
12 FROM dbo.sales
13 WHERE SaleID = 1001;
```

The 'Results' pane at the bottom shows the output of the query, which is a table with 9 rows and 2 columns: 'COLUMN_NAME' and 'DATA_TYPE'. The data is as follows:

Column Name	Data Type
SaleID	int
CustomerID	varchar
CustomerName	varchar
Region	varchar
Product	varchar
ProductCategory	varchar
OrderDate	date
Quantity	int
Revenue	float

The bottom status bar indicates that the query was successful, with a message: 'Succeeded (4 sec 336 ms)'. It also shows 'Copilot completions: Ready' and 'Code assistance: objects refreshed (refreshed 32m ago)'. The bottom right corner of the results pane shows 'Columns: 2 Rows: 9'.

SELECT * FROM dbo.sales WHERE SaleID = 1001 - baseline record

The screenshot shows the Microsoft Fabric SQL query editor interface. The left sidebar displays the Explorer pane with the following structure:

- sample_data
 - Schemas
 - dbo
 - Tables
 - sales
 - SaleID
 - Custo
 - Custo
 - Regio
 - Prodl
 - Prodl
 - Order
 - Quan
 - Rever
- Views
- Functions
- Stored Proce
- INFORMATION_S
- queryinsights

The main query editor shows the following SQL code:

```
-- STEP 09: Validate SQL Backend and Baseline Record
-- 1. Validate dbo.sales table schema
SELECT
    COLUMN_NAME,
    DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'dbo'
    AND TABLE_NAME = 'sales'
ORDER BY ORDINAL_POSITION;

-- 2. Validate the controlled test record
SELECT *
FROM dbo.sales
WHERE SaleID = 1001;
```

The bottom of the interface shows the Results pane with a table containing one row of data:

	SaleID	CustomerID	CustomerName	Region	Product	ProductCategory	OrderDate	Quantity	Revenue
1	1001	C001	Alice	Pune	Laptop	Electronics	2026-08-01	2	2400

10

STEP 10

Execute SQL Transaction with COMMIT

WHERE

Fabric Warehouse → SQL Query Editor

WHAT TO CLICK / OPEN

Open a new query window.

WHAT TO CREATE / SELECT / CONFIGURE

Run a controlled transaction against only SaleID = 1001 using existing writable columns.

WHAT TO VALIDATE

Transaction COMMIT control.

RECORD / EVIDENCE

Commit Result, SaleID, Quantity, Revenue.

RUN

```
BEGIN TRANSACTION;

UPDATE dbo.sales SET Quantity = Quantity + 1
WHERE SaleID = 1001;

UPDATE dbo.sales SET Revenue = Revenue + 0.01
WHERE SaleID = 1001;

COMMIT TRANSACTION;

SELECT SaleID, Quantity, Revenue
FROM dbo.sales WHERE SaleID = 1001;
```


The screenshot shows the Microsoft Fabric SQL query editor interface. The left sidebar displays the Explorer pane with a tree view of the 'sample_data' warehouse, including schemas, tables, and views. The main editor area shows a SQL query with two steps: 'STEP 10: Validate SQL Transaction COMMIT Control' and a query that updates the 'Sales' table for 'SaleID = 1001'. The query uses 'BEGIN TRANSACTION;', 'SELECT' to retrieve 'SaleID', 'Quantity', and 'Revenue', 'FROM dbo.sales', and 'WHERE SaleID = 1001;'. The results pane at the bottom shows a table with columns 'SaleID', 'Quantity', and 'Revenue', and a single row with values 1001, 2, and 2400 respectively.

```
1 -- STEP 10: Validate SQL Transaction COMMIT Control
2
3 BEGIN TRANSACTION;
4
5 SELECT
6     SaleID,
7     Quantity,
8     Revenue
9 FROM dbo.sales
10 WHERE SaleID = 1001;
11
12 COMMIT TRANSACTION;
13
14 SELECT
15     SaleID,
16     Quantity,
17     Revenue
18 FROM dbo.sales
19 WHERE SaleID = 1001;
```

	123 SaleID	123 Quantity	12 Revenue
1	1001	2	2400

11

STEP 11

Prove ROLLBACK Atomicity

WHERE

Fabric Warehouse → SQL Query Editor

WHAT TO CLICK / OPEN

Open a new query window.

WHAT TO CREATE / SELECT / CONFIGURE

Make a temporary change to SaleID 1001, inspect it, then roll it back.

WHAT TO VALIDATE

Values after ROLLBACK match the values before the transaction - nothing persisted.

RUN

```
BEGIN TRANSACTION;

-- Inspect the test record inside the
transaction
SELECT SaleID, Quantity AS QuantityInsideTx,
Revenue
FROM dbo.sales WHERE SaleID = 1001;

ROLLBACK TRANSACTION;

-- Verify the record after rollback
SELECT SaleID, Quantity AS
QuantityAfterRollback, Revenue
FROM dbo.sales WHERE SaleID = 1001;
```

11 ROLLBACK script - inside-transaction and after-rollback results for SaleID 1001

The screenshot shows the Microsoft Fabric SQL query editor interface. The left sidebar displays the Explorer pane with the 'sample_data' warehouse selected, showing a schema 'dbo' with a table 'sales'. The main editor area shows a SQL query titled 'SQL query 1' with the following script:

```
1 -- STEP 11: Validate SQL ROLLBACK Control
2
3 BEGIN TRANSACTION;
4
5 -- Inspect the controlled test record inside the transaction
6 SELECT
7     SaleID,
8     Quantity AS QuantityInsideTx,
9     Revenue
10 FROM dbo.sales
11 WHERE SaleID = 1001;
12
13 -- Roll back the transaction
14 ROLLBACK TRANSACTION;
15
16 -- Verify the record after rollback
17 SELECT
18     SaleID,
19     Quantity AS QuantityAfterRollback,
20     Revenue
21 FROM dbo.sales
22 WHERE SaleID = 1001;
```

Below the query editor, the 'Results' tab is active, displaying a table with the following data:

123 SaleID	123 QuantityInsideTx	12 Revenue
1 1001	2	2400



STEP 12

Validate Fabric API for GraphQL Schema

WHERE

Fabric Workspace → Fabric API for GraphQL

WHAT TO CLICK / OPEN

Open the GraphQL API → Schema / Docs / Explorer.

WHAT TO CREATE / SELECT / CONFIGURE

Locate the sales entity and inspect query fields, mutation fields, input types, return fields, SaleID and available writable fields.

WHAT TO VALIDATE

Confirm the actual sales entity and record the real mutation name and input structure exposed by the environment. Do not invent a mutation name.

RECORD / EVIDENCE

Entity, Mutation, Input, Return Fields, SaleID Field, Writable Field.

The screenshot displays the Microsoft Fabric GraphQL interface. At the top, a navigation bar includes tabs for 'sample_data', 'Sales_DirectLake_Model', and 'GraphQL_1'. A search bar and a 'Buy Fabric' button are also present. Below the navigation bar, a warning message states: 'Your free Microsoft Fabric trial ends in 0 days. After the trial ends, you won't be able to work with Fabric items you created during the trial unless you buy Microsoft Fabric or move them to a Power BI Premium capacity. [Learn more](#)'. The main interface is divided into three sections: 'Schema explorer', 'Query editor', and 'Query variables'. The 'Schema explorer' on the left shows a tree view with 'sample_data' expanded, containing 'sales', 'Queries', and 'Mutations'. The 'Query editor' in the center shows a GraphQL query:

```
1 query {  
2   sales(first: 10) {  
3     items {  
4       SaleID  
5     }  
6   }  
7 }
```

 The 'Query variables' section on the right shows a list of variables:

```
1  
2 {  
3  
4 }  
5
```

 The 'Results' section at the bottom shows a single result:

```
1
```



STEP 13

Validate GraphQL Write Capability

WHERE

Fabric API for GraphQL → Schema Explorer

WHAT TO CLICK / OPEN

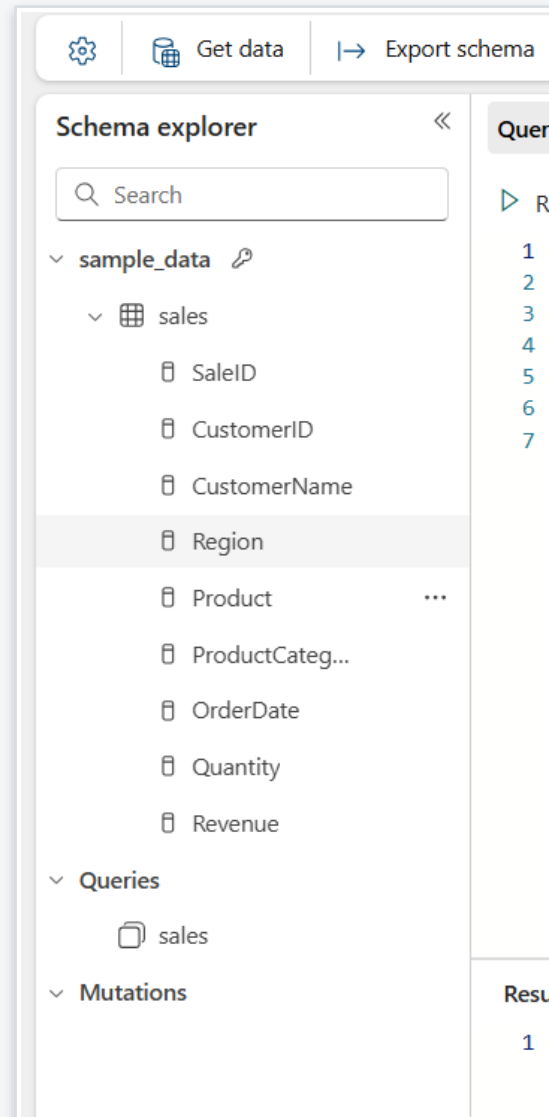
Open the GraphQL API and expand Mutations.

WHAT TO CREATE / SELECT / CONFIGURE

Verify whether a mutation is exposed for the sales entity. The environment exposes sales under Queries, but no mutation is available.

WHAT TO VALIDATE

Mutations node is empty - the sales entity is read-only through GraphQL in this environment.



14

STEP 14

Validate Fabric App Write-Back Configuration

WHERE

Fabric Workspace → Apps → Sales Operations → Sales Order Approvals

WHAT TO CLICK / OPEN

Open the Sales Order Approvals app and open the Order Approvals screen.

WHAT TO CREATE / SELECT / CONFIGURE

Locate record 1001 and verify: Record ID = 1001 · Order Status = Pending · Approve Order action is visible. Then open the app in Edit mode, select Approve Order and inspect its OnSelect formula.

WHAT TO VALIDATE

Confirm whether the approval action has an actual configured write-back / backend operation.

14

Order Approvals screen - record 1001, OrderStatus Pending, Approve Order visible

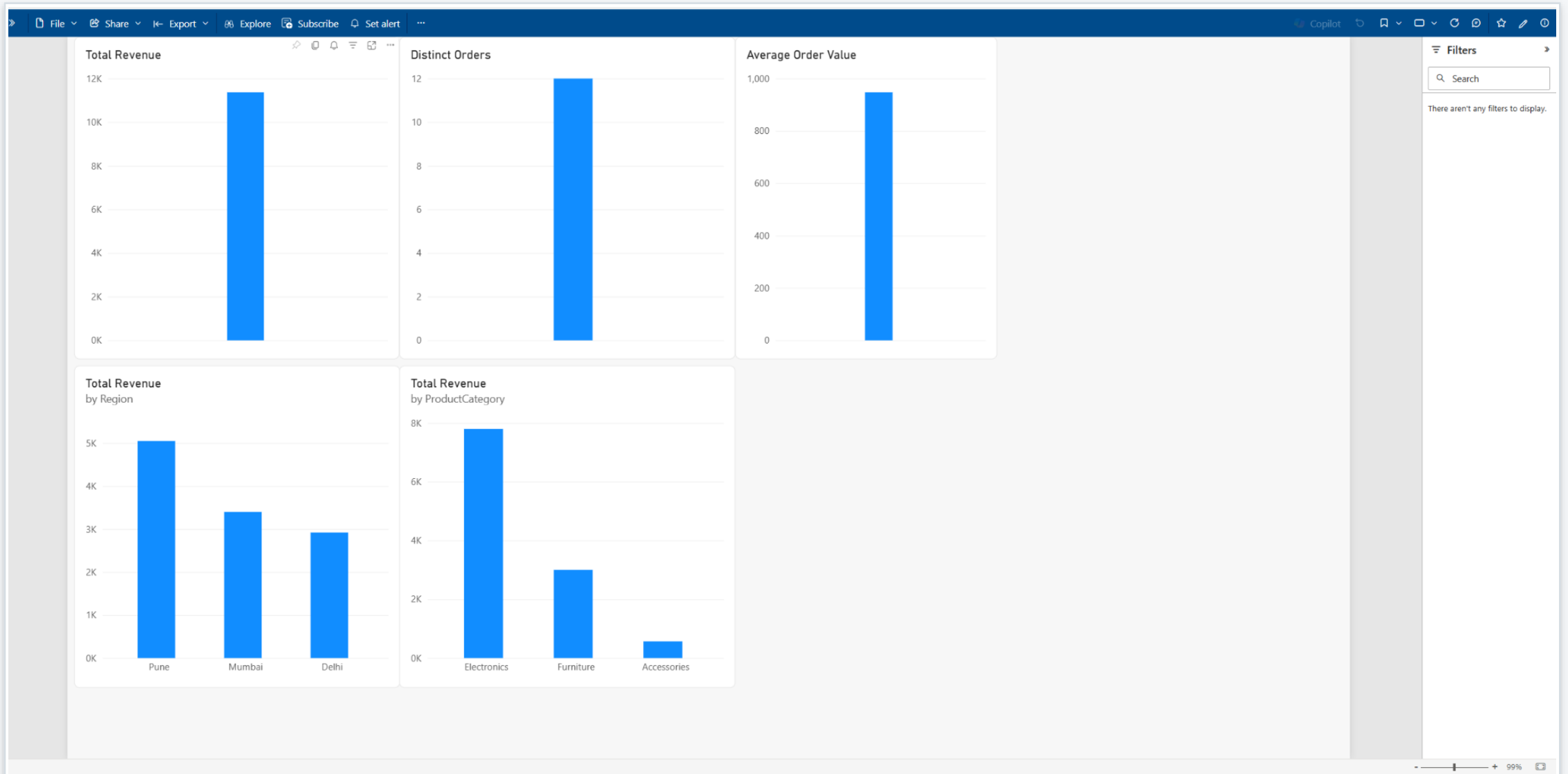
Power Apps | Sales Order Approvals (Editing) <https://academy.langchain.com/courses/deep-agents-with-langgraph> Environment KloudKraft

1001

OrderStatus: Pending

ApprovedBy:

Approve Order



Messages

Results

Open in Excel

Explore this data (preview)

Copy

Search

	123 SaleID	ABC CustomerID	ABC CustomerName	ABC Region	ABC Product	ABC ProductCategory	OrderDate	123 Quantity	12 Revenue
1	1001	C001	Alice	Pune	Laptop	Electronics	2026-08-01	2	2400

15

STEP 15

Validate Event-Driven Write-Back using Eventstream

WHERE

Fabric Workspace → Eventstream

WHAT TO CLICK / OPEN

Open the assigned Eventstream, inspect Source / Input, Transformation and Destination / Sink, then open Data Preview or monitoring.

WHAT TO CREATE / SELECT / CONFIGURE

Use dbo.sales and controlled test record SaleID 1001. Trigger one controlled SQL update.

WHAT TO VALIDATE

SQL update succeeds, an event for SaleID 1001 is captured, and it is routed to the configured destination.

RECORD / EVIDENCE

Eventstream, Source/Input, Test Record, Changed Field, Event Captured PASS/FAIL, Destination/Sink, Event Routed PASS/FAIL, Timestamp.

Detailed sub-steps with screenshots follow →

15.1 Create New Warehouse

- Click labskraft-data2 on the far-left rail to view your workspace
- Click + New item (top left) and select Warehouse

15

15.2 Name the Warehouse

- Enter a name, e.g. Sales_Warehouse
- Click Create

Screenshot on the next slide →

New Warehouse dialog - name entered as Sales_Warehouse

The screenshot displays the Microsoft Fabric interface. At the top, there's a navigation bar with tabs for 'sample_data', 'Sales_Eventstream', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', and 'Sales_GraphQL_Write_Test'. The main workspace area shows a list of items under the 'labskraft-data2' workspace. A modal dialog titled 'New Warehouse' is open in the center, allowing the user to create a new warehouse. The dialog includes a 'Name' field with the value 'Sales_Warehouse' and a 'Location' dropdown menu set to 'labskraft-data2'. There are 'Advanced', 'Create', and 'Cancel' buttons at the bottom of the dialog. The background workspace contains a table with columns: Name, Status, Next refresh, Endorsement, Sensitivity, and Included in app. The table lists various items including 'synapsevolks_Migration', 'amitpoc', and several notebooks.

Name	Status	Next refresh	Endorsement	Sensitivity	Included in app
synapsevolks_Migration		—	—	—	
amitpoc		—	—	—	
fabricpocamit_de13c963-5c38-408d-99e1-e90cab46d713		—	—	—	
GraphQL_1	API for GraphQL	—	Kunal Narkhede	—	
manish	Notebook	—	VolksWagen U13	—	
manish test	Notebook	—	VolksWagen U14	—	
nb_bronze_ingest	Notebook	—	VolksWagen U13	—	
nb_gold_aggregate	Notebook	—	VolksWagen U13	—	
Notebook 1	Notebook	—	VolksWagen U13	—	
Notebook 2	Notebook	—	VolksWagen U14	—	
Notebook 3	Notebook	—	VolksWagen U15	—	

15.3 Copy Table into Warehouse

- Open a New SQL query in the new Warehouse
- Create and populate `dbo.sales` from the sample data:

RUN

```
CREATE TABLE dbo.sales AS  
SELECT * FROM sample_data.dbo.sales;
```

Screenshot on the next slide →

CREATE TABLE AS SELECT completed in Sales_Warehouse

The screenshot displays the Microsoft Fabric web interface. The top navigation bar includes tabs for various workspaces: sample_data, sample_data, Sales_Eventstream, Sales_DirectLake_Model, Sales_Business_Terminology_Report, Sales_GraphQL_Write_Test, and Sales_Warehouse. The left sidebar shows the Explorer view with a tree structure of the Sales_Warehouse, including Schemas, dbo, Tables, and sales. The main area shows a SQL query editor with a single query: `CREATE TABLE dbo.sales AS SELECT * FROM sample_data.dbo.sales;`. The query has been executed successfully, as indicated by the 'Messages' pane at the bottom, which shows a log of the execution process. The status bar at the bottom indicates that the query succeeded in 2 seconds and 791 milliseconds.

Explorer

- Warehouses
 - Sales_Warehouse
 - Schemas
 - dbo
 - Tables
 - sales
 - 123 SaleI
 - ABC Custo
 - ABC Custo
 - ABC Region
 - ABC Prodl
 - ABC Prodl
 - Order
 - 123 Quan
 - 12 Rever
 - Views
 - Functions
 - Stored Proce
 - INFORMATION_S
 - sys
 - Security

SQL query 1

```
1 CREATE TABLE dbo.sales AS SELECT * FROM sample_data.dbo.sales;
```

Messages

- 9:33:10 PM Started executing on line 1
Statement ID: {5063CF4F-4973-4E55-AA1E-6C709BA22007} | Query hash: 0x2C619F598F1AEB5F | Distributed request ID: {71E68D5A-CC8D-40B1-8699-32A64DFB0168}
(12 records affected)
- 9:33:13 PM Query execution time: 00:00:02.014 | Time to process and render results: 00:00:00.777 | Total duration: 00:00:02.791

AutoSave: On (Last saved: 1m ago) | Succeeded (2 sec 791 ms) | Copilot completions: On | Copilot completions: Ready | Code assistance: objects refreshed

15

15.4 Run the Update

- Run the controlled update inside the new Warehouse:

RUN

```
UPDATE dbo.sales SET Revenue = Revenue + 0.01  
WHERE SaleID = 1001;
```

Screenshot on the next slide →

UPDATE on SaleID 1001 executed - Messages pane confirms 1 row affected

The screenshot displays the Microsoft Fabric SQL query editor interface. The top navigation bar includes tabs for various workspaces: sample_data, sample_data, Sales_Eventstream, Sales_DirectLake_Model, Sales_Business_Terminology_Report, Sales_GraphQL_Write_Test, and Sales_Warehouse. The left sidebar shows the Explorer pane with a tree view of the Sales_Warehouse database structure, including Schemas, Tables, and Views. The main editor area shows a SQL query in 'SQL query 2' tab: `UPDATE dbo.sales SET Revenue = Revenue + 0.01 WHERE SaleID = 1001;`. Below the query editor, the Messages pane displays the execution results, confirming that 1 record was affected. The status bar at the bottom indicates that the query was successful and that Copilot completions are ready.

SQL query 2

```
UPDATE dbo.sales SET Revenue = Revenue + 0.01 WHERE SaleID = 1001;
```

Messages

9:35:12 PM Started executing on line 1
Statement ID: {48ABE930-F0F7-4A82-B357-03CB08A30413} | Query hash: 0xD59CAC821636408C | Distributed request ID: {4203DE5B-48D4-4D89-B776-0310C38A4F53}
(1 records affected)

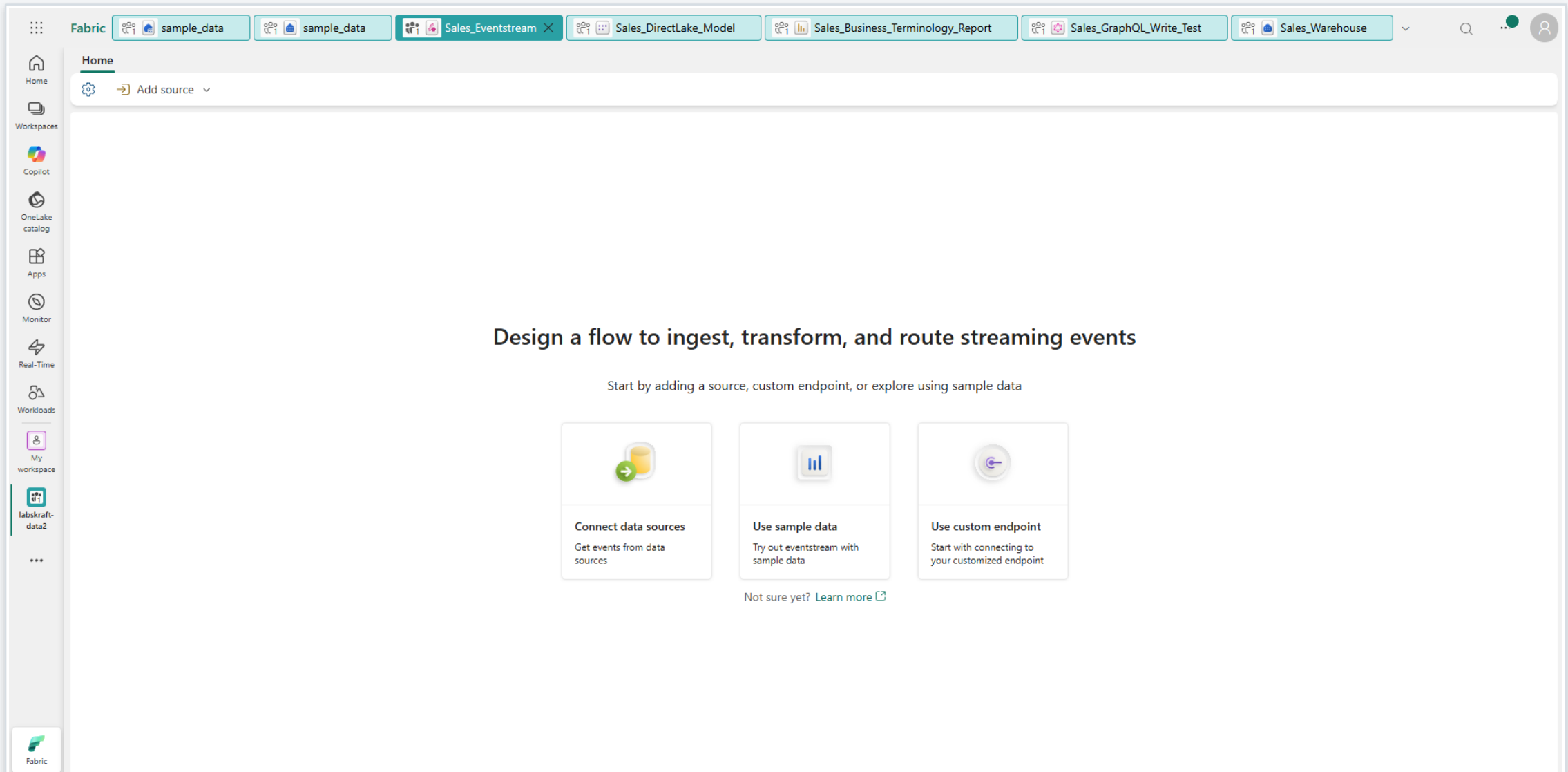
9:35:17 PM Query execution time: 00:00:03.305 | Time to process and render results: 00:00:00.851 | Total duration: 00:00:04.156

AutoSave: On (Saved) Succeeded (4 sec 156 ms) Copilot completions: On Copilot completions: Ready Code assistance: objects refreshed (refreshed 1m ago)

15.5 Create / Open Eventstream

- Click + New item, select Eventstream
- Name it Sales_Eventstream (or open the existing one from the item list)

Screenshot on the next slide →



15.6 Add Source

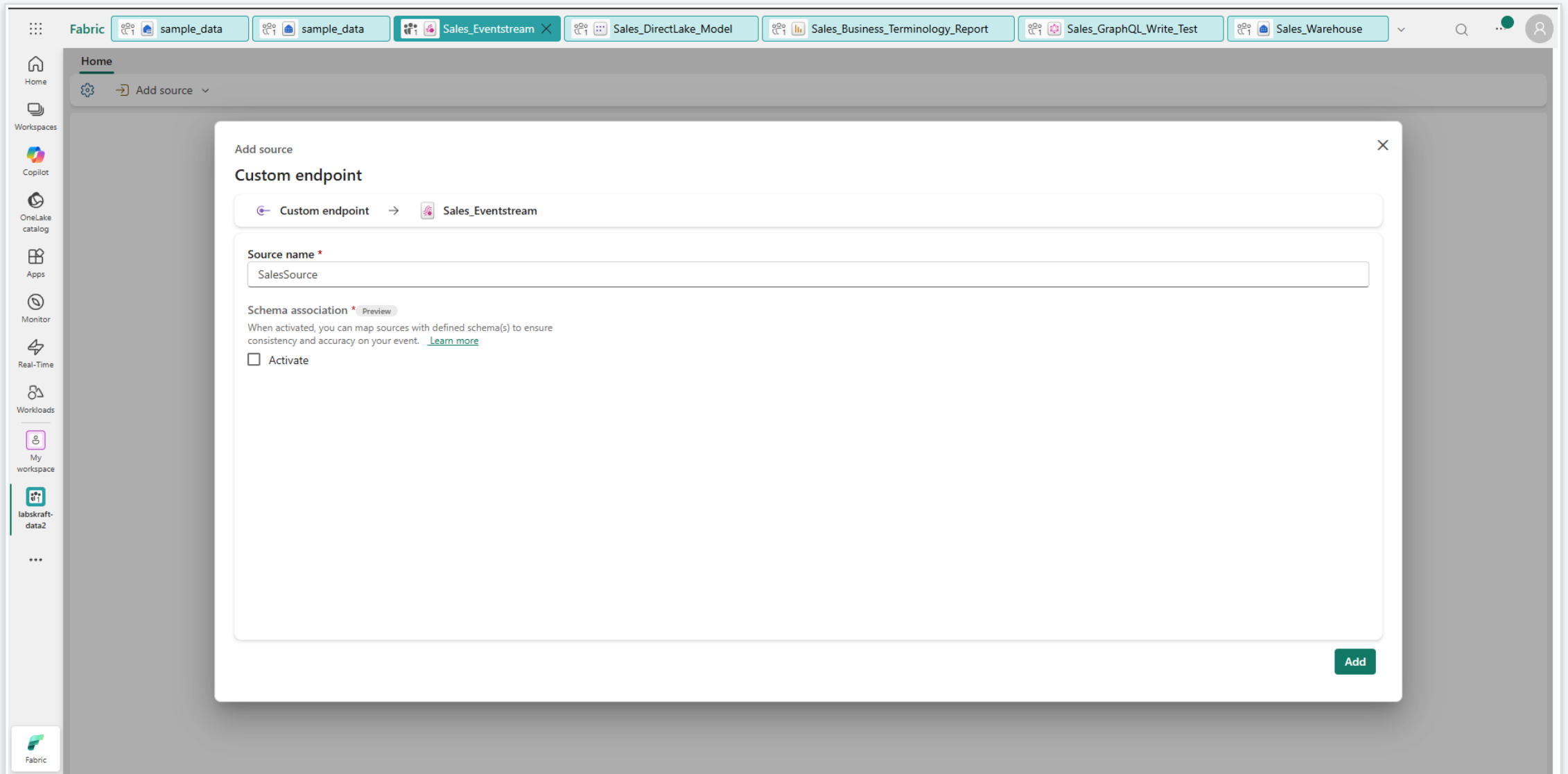
- Click Use custom endpoint, or + Add source in the top menu
- Select Custom App or Fabric Item / Database as directed

15

15.7 Configure Source Node

- Enter a source name, e.g. SalesSource
- Click Add

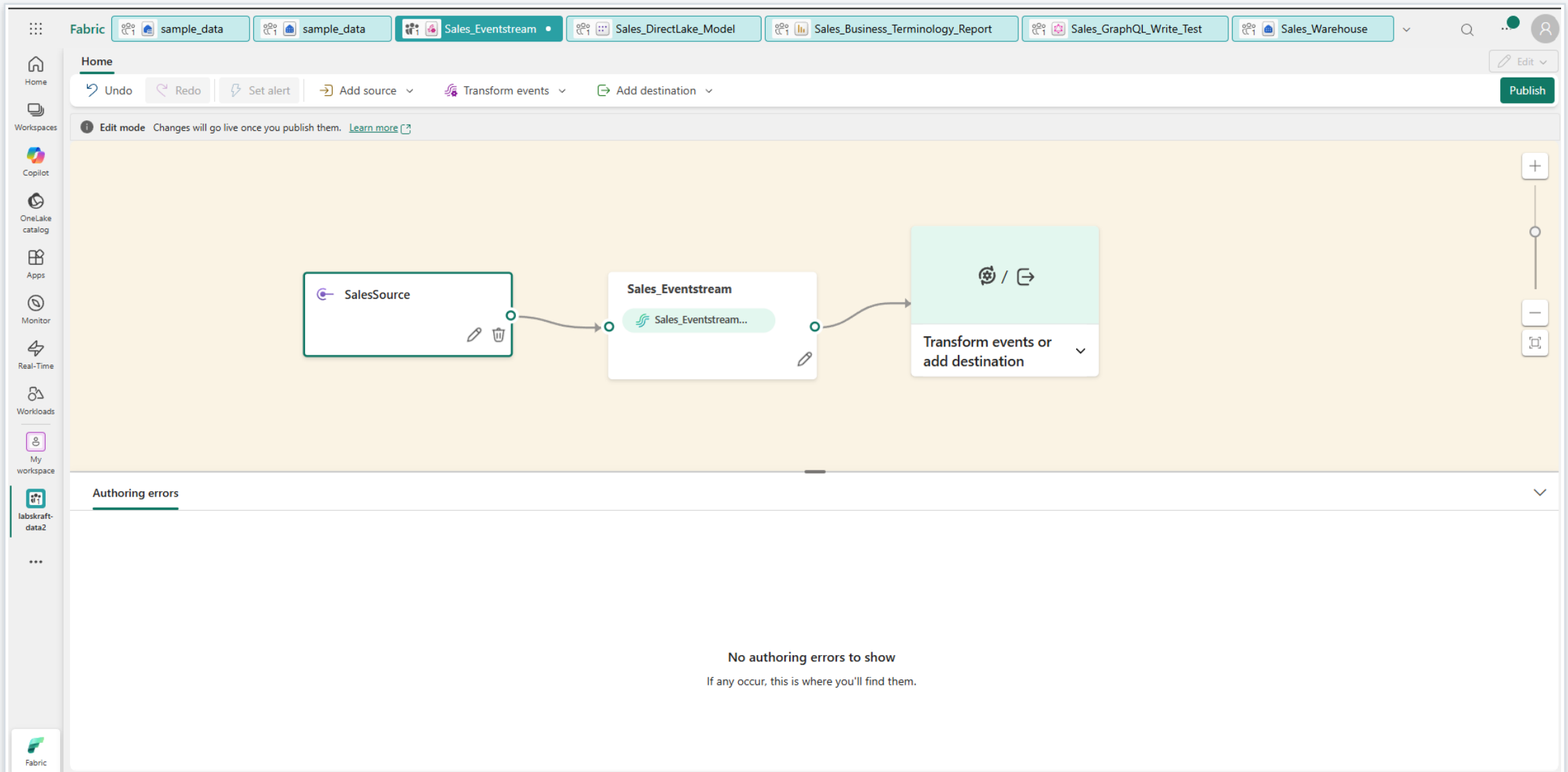
Screenshot on the next slide →



15.8 Verify Stream Canvas

- The SalesSource node appears next to the Sales_Eventstream node
- Click Publish

Screenshot on the next slide →



15.9 Copy Connection Keys & Publish

- Click the SalesSource node
- Copy the Event hub name and the Connection string-primary key

Screenshot on the next slide →

SalesSource node - Event hub name and connection string keys shown in the details pane

The screenshot displays the Microsoft Fabric Data Studio interface. The top navigation bar shows several workspaces: sample_data, sample_data, Sales_Warehouse, Sales_DirectLake_Model, Sales_Business_Terminology_Report, Sales_GraphQL_Write_Test, and Sales_Eventstream. The left sidebar contains navigation icons for Home, Workspaces, Copilot, OneLake catalog, Apps, Monitor, Real-Time, Workloads, My workspace, and labskraft-data2. The main canvas shows a data pipeline with three nodes: SalesSource (Active), Sales_Eventstream (Sales_Eventstream...), and a final destination node. The SalesSource node is highlighted with a green border. Below the canvas, the 'Details' pane is open, showing the configuration for the SalesSource node. The 'Protocol' is set to 'Event Hub'. The 'Basic' tab is selected, displaying the following details:

Property	Value
Event hub name	esehpn0mpqcos5n8azqxo8_eh
Shared access key name	key_6ccf8f27-b39e-4d1f-95b1-d01a31c649f0
Primary key	
Secondary key	
Connection string-primary key	Endpoint=sb://esehpn0mpqcos5n8azqxo8.servicebu
Connection string-secondary key	

A 'Show sample code' button is visible in the top right corner of the details pane.

15.10 Add the Event Hubs Connector to Power Apps

- In Power Apps Studio click the Data icon (cylinder) on the left panel
- Click + Add data, search Azure Event Hubs and select it
- Choose Connect directly and paste the Connection string-primary key from the SalesSource node
- Click Connect

15.11 Update OnSelect of the Approve Button

- Select the Approve Order button
- Ensure OnSelect is selected in the property dropdown
- Replace the formula with the Event Hubs send command (use your exact Event hub name):

RUN

```
EventHubs.SendEvent(  
    "<your_event_hub_name>",  
    { ContentData: "{ \"\"OrderID\": 1001,  
        \"\"OrderStatus\": \"Approved\",  
        \"\"ApprovedBy\": \"User\"}\" }"  
    );  
UpdateContext({OrderStatus: "Approved"})
```

Screenshot on the next slide →

The screenshot displays the Power Apps Studio interface for editing a canvas app named "Sales Order Approvals". The top navigation bar shows the "Environment" as "KloudKraft". The left sidebar contains a "Data" pane with a search bar and an "Add data" button, and a "Components" pane showing an "EventHubs" component. The main canvas area shows a button labeled "Approve Order" with a tooltip that reads "OrderStatus: Pending" and "ApprovedBy: Approve Order". The formula bar at the top shows the "OnSelect" formula: `EventHubs.SendEvent("esehpn0mpqcos5n8azqxo8_eh", { ContentData: "{\"OrderID\": 1001, \"OrderStatus\": \"Approved\", \"ApprovedBy\": \"User\"} }); UpdateContext({OrderStatus: \"Approved\"})`. A context menu is open over the formula bar, offering options: "Format text", "Remove formatting", and "Find and replace". The bottom status bar indicates the current view is "Order Approvals" and the selected element is "Button2".

Power Apps | Sales Order Approvals (Editing)

Environment: KloudKraft

Back | Undo | Redo | Insert | Open Sans | 15 | Editing | Publish

OnSelect = `EventHubs.SendEvent("esehpn0mpqcos5n8azqxo8_eh", { ContentData: "{\"OrderID\": 1001, \"OrderStatus\": \"Approved\", \"ApprovedBy\": \"User\"} }); UpdateContext({OrderStatus: \"Approved\"})`

Format text | Remove formatting | Find and replace

OrderStatus: Pending

ApprovedBy: Approve Order

Order Approvals | Button2

50%

15.12 Trigger the Write-Back Action

- Click Play (▶) in the top-right, or press F5
- Click Approve Order
- Save and Publish the Power App

Screenshot on the next slide →

Power Apps | Sales Order Approvals (Editing)

Environment
KloudKraft

1001

OrderStatus: Pending

ApprovedBy:

Approve Order

✓ Publish successful
9/5/2026, 10:59:58 PM
Sales Order Approvals is now available to everyone.

Mobile view icons: Desktop, Tablet, Mobile, Print

15.13 Capture and Validate in Eventstream

- Return to the Sales_Eventstream tab in Fabric
- Click the Sales_Eventstream centre node
- Bottom pane → Data preview → Refresh
- A row shows OrderID / SaleID 1001 with OrderStatus Approved

Screenshot on the next slide →

15 Eventstream Data preview - captured event for record 1001 with OrderStatus Approved

The screenshot displays the Microsoft Fabric interface, specifically the 'Sales_Eventstream' workspace. The top navigation bar shows several tabs: 'sample_data', 'sample_data', 'Sales_Warehouse', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', 'Sales_GraphQL_Write_Test', and 'Sales_Eventstream'. The left sidebar contains a 'Data' section with a tree view showing 'Sources' (SalesSource), 'Streams' (Sales_Eventstream-...), 'Operators' (No Operators), and 'Destinations' (No Destinations). The main canvas displays a data flow diagram with a 'SalesSource' node connected to a 'Sales_Eventstream' node, which is then connected to a destination node. A tooltip for the destination node reads: 'Switch to edit mode to Transform event or add destination'. The bottom section shows the 'Data preview' tab with a table of captured events. The table has columns: OrderID, OrderStatus, and ApprovedBy. The data shows two records for OrderID 1001, both with OrderStatus 'Approved' and ApprovedBy 'User'. The 'Last refreshed' timestamp is 09/05/26 11:15:44 PM.

Data preview | Data insights | Refresh

Hide settings | Search | Show details

OrderID	OrderStatus	ApprovedBy
1001	Approved	User
1001	Approved	User

Last refreshed: 09/05/26 11:15:44 PM



STEP 16

Run Concurrent SQL Write Test

WHERE

Fabric Warehouse → SQL Query Editor

WHAT TO CLICK / OPEN

Open two independent SQL query sessions.

WHAT TO CREATE / SELECT / CONFIGURE

Use SaleID 1001 only.

WHAT TO VALIDATE

Observe actual concurrency behaviour and confirm no duplicate or corrupt record.

RECORD / EVIDENCE

Session A, Session B, blocked/succeeded/failed, wait, final quantity, deadlock, data corruption, concurrency result.

Detailed sub-steps with screenshots follow →

16.1 Open two query tabs

- Go to the Sales_Warehouse tab in Fabric
- Click New SQL query twice - Session A and Session B side by side

16.2 Run Session A (exclusive lock)

- In Session A, run the in-flight, delayed transaction:

RUN

```
UPDATE dbo.sales SET Quantity = Quantity + 10  
WHERE SaleID = 1001;
```

Screenshot on the next slide →

The screenshot displays the Microsoft Fabric SQL interface. At the top, a navigation bar includes tabs for 'sample_data', 'Sales_Warehouse', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', 'Sales_GraphQL_Write_Test', and 'Sales_Eventstream'. Below this, a 'Home' tab is active, showing a toolbar with options like 'Get data', 'New SQL query', 'SQL templates', 'Monitor', 'New semantic model', 'New API for GraphQL', 'Add to data agent', 'Download SQL database project', 'Open in', and 'Copilot'. The left sidebar contains an 'Explorer' pane with a tree view showing 'Warehouses' (Sales_Warehouse), 'Schemas', 'Security', 'Queries' (My queries, Session A, Session B, SQL query 1, SQL query 2), and 'Shared queries'. The main workspace shows 'Session A' selected, with a toolbar for 'Run', 'Save as', 'Save as view', 'Copy query', 'Create rule', 'Explain query', and 'Fix query errors'. A preview message states: 'Copilot uses AI. Mistakes can happen. Verify code suggestions before running them. Review terms'. The SQL editor contains the following query:

```
1 UPDATE dbo.sales SET Quantity = Quantity + 10 WHERE SaleID = 1001;
```

At the bottom, a 'Messages' pane shows the execution log:

- 11:33:17 PM: Started executing on line 1
Statement ID: {D4DD35C3-9AAE-4ABD-ADD0-FBCC2E468C18} | Query hash: 0xE4D11AA88A2CC190 | Distributed request ID: {2E19D637-00CE-4BA5-8A3C-103E289E5654}
(1 records affected)
- 11:33:22 PM: Query execution time: 00:00:04.219 | Time to process and render results: 00:00:01.071 | Total duration: 00:00:05.290

16.3 Run Session B (concurrent write)

- Immediately switch to Session B while A is executing and run:

RUN

```
UPDATE dbo.sales SET Quantity = Quantity + 20  
WHERE SaleID = 1001;
```

Screenshot on the next slide →

16 Session B - concurrent UPDATE; note the conflict message returned by the Warehouse

The screenshot displays the Microsoft Fabric SQL interface. The top navigation bar includes tabs for various data sources: sample_data, sample_data, Sales_Warehouse (active), Sales_DirectLake_Model, Sales_Business_Terminology_Report, Sales_GraphQL_Write_Test, and Sales_Eventstream. The left sidebar shows the Explorer view with a tree structure under 'Sales_Warehouse' containing 'Schemas', 'Security', and 'Queries'. The 'Queries' folder is expanded, showing 'My queries' with 'Session A' and 'Session B' (selected). Below 'My queries' are 'SQL query 1', 'SQL query 2', and 'Shared queries'. The main workspace shows the SQL editor for 'Session B' with a single query: `UPDATE dbo.sales SET Quantity = Quantity + 20 WHERE SaleID = 1001;`. The 'Messages' pane at the bottom displays the execution results, including a conflict message: 'Snapshot isolation transaction aborted due to update conflict. Using snapshot isolation to access table 'sales' directly or indirectly in database 'Sales_Warehouse' can cause update conflicts if rows in that table have been deleted or updated by another concurrent transaction. Retry the transaction.'

Fabric

sample_data sample_data Sales_Warehouse Sales_DirectLake_Model Sales_Business_Terminology_Report Sales_GraphQL_Write_Test Sales_Eventstream

Home Management Help

Analyze data with Share

Get data New SQL query SQL templates Monitor New semantic model New API for GraphQL Add to data agent Download SQL database project Open in Copilot

Explorer

Warehouses

Sales_Warehouse

Schemas

Security

Queries

My queries

Session A

Session B

SQL query 1

SQL query 2

Shared queries

Run Save as Save as view Copy query Create rule Explain query Fix query errors

Preview Copilot uses AI. Mistakes can happen. Verify code suggestions before running them. Review terms

1 UPDATE dbo.sales SET Quantity = Quantity + 20 WHERE SaleID = 1001;

Messages Copy messages

11:33:20 PM Started executing on line 1
Msg 24556, Level 16, State 2, Line 1
Snapshot isolation transaction aborted due to update conflict. Using snapshot isolation to access table 'sales' directly or indirectly in database 'Sales_Warehouse' can cause update conflicts if rows in that table have been deleted or updated by another concurrent transaction. Retry the transaction.

11:33:22 PM Query execution time: 00:00:01.189 | Time to process and render results: 00:00:00.738 | Total duration: 00:00:01.927

16.4 Verify final state

- Once both sessions finish, run in either session:

RUN

```
SELECT SaleID, Quantity, Revenue  
FROM dbo.sales WHERE SaleID = 1001;
```

Screenshot on the next slide →

Final state of SaleID 1001 after both sessions

The screenshot displays the Microsoft Fabric interface, showing the final state of SaleID 1001 after two sessions. The interface includes a top navigation bar with tabs for sample_data, Sales_Warehouse, Sales_DirectLake_Model, Sales_Business_Terminology_Report, Sales_GraphQL_Write_Test, and Sales_Eventstream. The left sidebar shows the Explorer with Warehouses, Schemas, Security, and Queries. The main pane displays SQL query 1 and SQL query 2, with Session B selected. The query results show a table with 3 columns: SaleID, Quantity, and Revenue. The table has 1 row with the following data: SaleID 1001, Quantity 42, and Revenue 2400.02. The bottom status bar shows AutoSave: On (Saved), Succeeded (2 sec 283 ms), Copilot completions: On, Copilot completions: Ready, and Code assistance: objects refreshed (refreshed 3m ago).

SQL query 1

```
UPDATE dbo.sales SET Quantity = Quantity + 20 WHERE SaleID = 1001;
```

SQL query 2

```
SELECT SaleID, Quantity, Revenue FROM dbo.sales WHERE SaleID = 1001;
```

123 SaleID	123 Quantity	12 Revenue
1 1001	42	2400.02

Columns: 3 Rows: 1



STEP 17

Run Concurrent GraphQL Write Test

WHERE

Fabric API for GraphQL → two independent client / query sessions

WHAT TO CLICK / OPEN

Open two GraphQL client / query windows with valid permissions.

WHAT TO CREATE / SELECT / CONFIGURE

Use the actual mutation from Step 12 and target only SaleID 1001 with different test values.

WHAT TO VALIDATE

Requests complete without unhandled server / deadlock errors.

RECORD / EVIDENCE

Client A result, Client B result, GraphQL errors, final value, serialized final state, concurrency result.

Detailed sub-steps with screenshots follow →

17.1 Open GraphQL editor

- Switch to the Sales_GraphQL_Write_Test tab in Fabric (or open your Fabric API for GraphQL item)

17 17.2 Attempt mutation (validate schema restriction)

- Paste and run the mutation against record 1001
- Observe the error confirming mutations are not exposed (read-only, as in Step 13)

RUN

```
mutation {  
  updateSales(SaleID: 1001, Quantity: 50) {  
    SaleID Quantity  
  }  
}
```

Screenshot on the next slide →

Mutation attempt - response pane returns error: mutations not supported for this schema

The screenshot displays the Microsoft Fabric web interface. At the top, a navigation bar includes the 'Fabric' logo and several workspace tabs: 'sample_data', 'sample_data', 'Sales_Warehouse', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', and 'Sales_GraphQL_Write_Test'. The left sidebar contains icons for Home, Workspaces, Copilot, OneLake catalog, Apps, Monitor, Real-Time, Workloads, My workspace, and a specific workspace 'labskraft-data2'. The main content area is divided into three sections. The top section, 'Schema explorer', shows a tree view with 'sample_data' expanded, containing 'sales' and 'Queries'. The middle section, 'Query1' and 'Query2', shows a GraphQL mutation query: `mutation { updateSales(SaleID: 1001, Quantity: 50) { SaleID Quantity } }`. The bottom section, 'Results', displays the response:

```
1 [
2   "errors": [
3     {
4       "message": "This GraphQL schema does not support `Mutation` operations."
5     }
6   ]
7 ]
```

. A status bar at the bottom indicates 'Failed (Total duration: 517 ms | Data source execution: 0 ms)'. The bottom navigation bar shows 'Schema' and 'Query' tabs.

17.3 Execute read query for record 1001

- Replace with a standard query and click Run (Play)

RUN

```
query {  
  sales(filter: { SaleID: { eq: 1001 } }) {  
    items { SaleID Quantity Revenue }  
  }  
}
```

Screenshot on the next slide →

Read query - persistent state of record 1001 returned in the results pane

The screenshot displays the Microsoft Fabric GraphQL interface. The top navigation bar includes tabs for 'sample_data', 'Sales_Warehouse', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', and 'Sales_GraphQL_Write_Test'. The left sidebar shows the 'Schema explorer' with a search bar and a tree view containing 'sample_data', 'Queries', and 'Mutations'. The main workspace is divided into three panes: 'Query1' and 'Query2' tabs at the top, and a 'Results' pane at the bottom. The 'Query1' pane shows a GraphQL query:

```
1 query {  
2   sales(filter: { SaleID: { eq: 1001 } }) {  
3     items {  
4       SaleID  
5       Quantity  
6       Revenue  
7     }  
8   }  
9 }
```

 The 'Query variables' pane on the right shows an empty object:

```
1 {  
2 }  
3  
4 }  
5
```

 The 'Results' pane shows the JSON response:

```
1 {  
2   "data": {  
3     "sales": {  
4       "items": [  
5         {  
6           "SaleID": 1001,  
7           "Quantity": 2,  
8           "Revenue": 2400  
9         }  
10      ]  
11    }  
12  }  
13 }
```

 The status bar at the bottom indicates 'Succeeded (Total duration: 1828 ms | Data source execution: 1654 ms)'. The bottom navigation bar shows 'Schema' and 'Query' tabs.

18

STEP 18

Final End-to-End Validation and Evidence Package

WHERE

Fabric App → GraphQL → SQL Warehouse → Eventstream → Semantic Model / Report

WHAT TO CLICK / OPEN

Keep the Sales Operations app, GraphQL API, SQL Query Editor, Eventstream and Sales_DirectLake_Model / report open.

WHAT TO CREATE / SELECT / CONFIGURE

Perform one final controlled validation using SaleID 1001 and trace: Fabric App → GraphQL API → SQL / Fabric Warehouse → Committed Write → Eventstream → Analytics / Semantic Model.

WHAT TO VALIDATE

Final record state is consistent across every surface.

RUN

```
SELECT * FROM dbo.sales WHERE SaleID = 1001;
```

Final verification - SELECT * on SaleID 1001 in the Warehouse

The screenshot displays the Microsoft Fabric interface. The top navigation bar includes tabs for 'sample_data', 'Sales_Warehouse', 'Sales_DirectLake_Model', 'Sales_Business_Terminology_Report', and 'Sales_GraphQL_Write_Test'. The left sidebar shows the 'Explorer' pane with a tree view of 'Sales_Warehouse' containing 'Schemas', 'Security', and 'Queries'. Under 'Queries', 'My queries' is expanded, showing 'Session A', 'Session B', 'SQL query 1', 'SQL query 2', and 'SQL query 3'. 'SQL query 3' is selected. The main pane shows the SQL query editor with the query: `SELECT * FROM dbo.sales WHERE SaleID = 1001;`. Below the editor, the 'Results' tab is active, displaying a table with 9 columns and 1 row. The table headers are: SaleID, CustomerID, CustomerName, Region, Product, ProductCategory, OrderDate, Quantity, and Revenue. The data row shows: 1, 1001, C001, Alice, Pune, Laptop, Electronics, 2026-08-01, 42, and 2400.02. The bottom status bar indicates 'AutoSave: On (Saved)', 'Succeeded (0 sec 984 ms)', and 'Copilot completions: On'.

	123 SaleID	ABC CustomerID	ABC CustomerName	ABC Region	ABC Product	ABC ProductCategory	OrderDate	123 Quantity	12 Revenue
1	1001	C001	Alice	Pune	Laptop	Electronics	2026-08-01	42	2400.02

Reference Results for the Supplied Dataset

11,370

Total Revenue

12

Distinct Orders

947.50

Average Order Value

Pune

Highest-performing Region - 5,050

Electronics

Top Product Category - 7,800

Final Coverage Check and Evidence to Retain

FINAL COVERAGE CHECK

- ☐ Consolidated semantic model created / validated in Direct Lake mode
- ☐ XMLA / Tabular Editor metadata and version governance validated
- ☐ Business terminology / synonym usability validated
- ☐ Fabric App used as write-back front end
- ☐ Fabric API for GraphQL schema validated
- ☐ GraphQL mutation executed and SQL persistence verified
- ☐ SQL backend transactional configuration validated
- ☐ COMMIT transaction validated
- ☐ ROLLBACK / ACID atomicity validated
- ☐ Concurrent SQL writes tested
- ☐ Concurrent GraphQL writes tested
- ☐ Eventstream change capture validated
- ☐ Final end-to-end App → GraphQL → SQL → Eventstream flow validated
- ☐ Direct Lake analytics results validated
- ☐ Database Mirroring intentionally excluded

EVIDENCE TO RETAIN

- Dataset / table load and schema validation
- Direct Lake semantic model
- XMLA / Tabular Editor version and governance metadata
- Measures and expected analytics results
- Fabric App OrderID 1001 write-back
- GraphQL schema and mutation response
- SQL COMMIT and ROLLBACK outputs
- Eventstream event / data preview
- Concurrent SQL and GraphQL results
- Final end-to-end validation

End of Lab 08

Retain all evidence items and submit the completed coverage checklist with your SaleID 1001 validation outputs.